TELECOM CUSTOMER CHURN PREDICTION AND ANALYTICS

Cloud Computing Course Project Report

A Django-based customer management and machine-learning application with a Docker-first cloud deployment design

Submitted by: Adithya Dev B S

Register Number: 26MCA0050

Programme / Semester: MCA Semester 1

Course: Cloud Computing

Faculty Guide: Prof Daphne Lopez

Academic Year: 2026–2027

Abstract

Telecom customer churn occurs when customers discontinue a service, reducing recurring revenue and potentially increasing customer-acquisition costs. This project presents a web-based Telecom Customer Churn Prediction and Analytics application that combines customer record management, dashboard summaries, and machine-learning-based risk estimation. The application is implemented using Django, with a relational data model for customer profiles, subscribed services, billing details, churn labels, prediction records, and agent activity. A scikit-learn Random Forest pipeline is used to estimate churn probability and classify customers into High, Medium, and Low risk groups.

The supplied Telco Customer Churn dataset contains 7,043 records and 21 columns. 1,869 customers are labelled as churned and 5,174 as retained, producing an observed churn rate of 26.54%. The model workflow uses numeric scaling, categorical one-hot encoding, a stratified 90:10 train-test split, and a Random Forest classifier. The cloud deployment uses Docker to package the application, Render as the web hosting platform, and PostgreSQL as the persistent relational database. This separation supports reproducible deployment and avoids storing important records in the container filesystem.

The project demonstrates the integration of web development, database management, machine learning, containerisation, and cloud hosting.

Keywords: customer churn, Django, Random Forest, PostgreSQL, Docker, Render, cloud computing, customer analytics.

Contents

- 1. Introduction
- 2. Problem Statement and Objectives
- 3. Literature Review
- 4. Dataset and Key Exploratory Findings
- 5. System Requirements and Technology Stack
- 6. System Architecture and Design
- 7. Database Design
- 8. Machine-Learning Methodology
- 9. Dockerisation and Cloud Deployment
- 10. Implementation and Testing
- 11. Key Findings and Decisions
- 12. Results and Discussion
- 13. Security, Limitations and Future Work
- 14. Conclusion
- References

1. Introduction

1.1 Background

Telecommunication companies operate in competitive markets where customers can change providers due to contract flexibility, price, service quality, support experience, and competing offers. Churn analytics uses historical customer information to estimate which customers may leave. A useful system combines model output with a clear interface so that staff can review records and prioritise retention activity.

1.2 Project overview

The project is a Django web application rather than a stand-alone machine-learning notebook. It provides authentication, dashboard summaries, customer search and filtering, customer record operations, churn prediction, and risk categories. The model pipeline is separated from the web views so that data preparation and prediction logic can be maintained independently.

1.3 Cloud computing relevance

Docker packages the runtime and dependencies into a repeatable image. A cloud web service makes the application available through a public HTTPS URL, while a managed PostgreSQL database stores records independently of the application container. This architecture is suitable for a course demonstration and introduces practical cloud concepts such as container images, environment variables, deployment logs, managed services, and persistence.

1.4 Scope

The system is an academic prototype using an educational telecom churn dataset. It supports customer review and risk estimation but does not make automatic decisions about customers. Model quality depends on the training data and must be measured using a held-out test set.

2. Problem Statement and Objectives

2.1 Problem statement

Customer information is distributed across profile, service, and billing attributes. Without a unified view and repeatable scoring process, retention teams may find it difficult to identify and prioritise customers who are more likely to churn. The project addresses this problem through a central customer dashboard and a machine-learning prediction workflow.

2.2 Objectives

- Maintain related customer, service, billing, churn and prediction records.
- Provide authenticated access to the dashboard and customer-management pages.
- Allow customer search, filtering by risk category, and paginated results.
- Train a supervised model from historical churn labels.
- Convert predicted probabilities into understandable risk categories.
- Package the application in Docker and deploy it to a cloud web service.
- Use a managed PostgreSQL database for persistent hosted records.
- Verify the deployed application through smoke tests.

2.3 Expected outcome

The expected outcome is a publicly accessible demonstration of the Django application running from a Docker image, with customer data stored in PostgreSQL and prediction results displayed through the web interface.

3. Literature Review

3.1 Lalwani et al. (2021) — Customer churn prediction system: a machine learning approach

Lalwani, Mishra, Chadha and Sethi proposed a multi-stage churn prediction process involving data preprocessing, feature analysis, feature selection using a gravitational search algorithm, model training, and evaluation. They compared multiple classifiers and used cross-validation for hyperparameter tuning. Their reported results found that AdaBoost and XGBoost achieved the strongest accuracy in their experiments, with AUC also used to compare classifiers. This paper is relevant because it treats churn prediction as a complete modelling workflow rather than simply applying one algorithm. The present project similarly prepares mixed customer features, trains a classifier, and evaluates the result using classification metrics.

3.2 Vafeiadis et al. (2015) — A comparison of machine learning techniques for customer churn prediction

Vafeiadis and colleagues compared popular machine-learning techniques on a public telecommunications dataset and investigated whether boosting improved model performance. Their study reported that boosted versions generally outperformed the corresponding non-boosted methods in their experimental setting. The paper demonstrates why model comparison and validation are important: a familiar algorithm is not automatically the best model for every churn dataset. In the present project, Random Forest is selected as a practical baseline, but comparison with boosting methods would be a useful extension.

3.3 Verbeke et al. (2011) — Building comprehensible customer churn prediction models with advanced rule induction techniques

Verbeke and colleagues emphasised that churn models should be judged not only by predictive accuracy but also by comprehensibility and justifiability. They explored rule-induction approaches to build models whose patterns could be understood by business users. This is important for retention work because a risk score alone does not explain what staff should investigate. The current application presents risk categories in a dashboard; a future version could add feature explanations or interpretable rules.

3.4 Xie et al. (2009) — Customer churn prediction using improved balanced random forests

Xie and colleagues studied class imbalance and proposed an improved balanced Random Forest approach. Their work highlights that churn datasets often contain fewer churners than non-churners, which can make ordinary accuracy misleading. The supplied dataset in this project has a minority churn class, so class weighting and metrics such as recall, precision, F1-score, and ROC-AUC are relevant. The project uses class_weight=balanced in its Random Forest configuration, though further validation is needed to determine whether this is optimal.

3.5 Imani et al. (2020–2024 review)

Imani and colleagues conducted a systematic review of recent customer-churn research, examining machine-learning and deep-learning studies published between 2020 and 2024. Their review describes the continuing use of ensemble methods such as XGBoost and LightGBM, alongside growing use of deep-learning methods such as LSTM and CNN. It also discusses persistent challenges in evaluation, data characteristics, and model deployment. This review provides a current research overview and supports the decision to treat the present Random Forest model as a baseline rather than a final solution.

3.6 Literature review summary

The reviewed studies collectively suggest four lessons: preprocess data carefully; compare models rather than assume one is best; account for class imbalance; and balance predictive performance with interpretability. This project applies a pipeline-based Random Forest approach and provides a web interface for using predictions. Future evaluation should compare alternative models and provide clear explanations for risk estimates.

4. Dataset and Key Exploratory Findings

The supplied CSV contains one record per customer and includes demographic, service, contract, payment, tenure, charge, and churn-label fields.

Dataset measure	Observed value
Rows	7,043
Columns	21
Churn = Yes	1,869 (26.54%)
Churn = No	5,174 (73.46%)
Median tenure	29 months
Mean monthly charge	$64.76

4.1 Churn by contract type

Contract type	Observed churn rate
Month-to-month	42.7%
One year	11.3%
Two year	2.8%

Month-to-month customers have a higher observed churn rate than customers on one-year or two-year contracts. This is an association in the supplied data, not proof that contract type alone causes churn.

4.2 Churn by internet service

Internet service	Observed churn rate
DSL	19.0%
Fiber optic	41.9%
No	7.4%

Churn rates differ across internet-service groups. These differences can help formulate questions for retention analysis, but they may be influenced by tenure, contract type, pricing, and other variables.

5. System Requirements and Technology Stack

5.1 Functional requirements

- User login and access control for protected pages.
- Dashboard counts and churn summaries, including contract-level views.
- Customer search, risk filtering, active/archived filtering and pagination.
- Customer profile maintenance and controlled archive/restore actions.
- Prediction of churn probabilities for eligible customers.
- Storage of risk categories and prediction timestamps.
- Administrative agent management and selected activity auditing.

5.2 Non-functional requirements

- Maintainable separation of views, models, templates and ML logic.
- Consistent local and cloud execution through Docker.
- Persistent relational storage through PostgreSQL.
- Secure environment-based configuration and restricted access.
- Usable risk labels and dashboard summaries.
- Repeatable model training and evaluation with documented dependencies.

5.3 Technology stack

Component	**Technology**	**Purpose**
Backend	Python, Django	Routes, authentication, forms, ORM and views
Frontend	Django templates, HTML/CSS	Dashboard and customer workflows
Machine learning	pandas, scikit-learn, joblib	Preprocessing, training and inference
Classifier	Random Forest	Churn probability estimation
Database	PostgreSQL	Persistent structured records
Container	Docker	Portable application image
Cloud web host	Render	Public web service
Static files	WhiteNoise / Django static files	Serving collected static assets

6. System Architecture and Design

6.1 Architecture overview

The application uses a simple three-part deployment architecture: the user accesses the Django application through a browser; the Django container handles authentication, views, templates and model inference; and a managed PostgreSQL database stores customer and prediction records.

Component	Responsibility
Browser	Displays pages and sends HTTPS requests
Render web service (Docker container)	Runs Django application and prediction workflow
Django application	Authentication, validation, customer operations and dashboard
Machine-learning pipeline	Transforms customer fields and estimates churn probability
PostgreSQL database	Stores customer-related records and prediction results

6.2 Request and prediction flow

1. User opens the deployed HTTPS URL and signs in.
2. Django validates credentials and opens an authenticated session.
3. The dashboard queries customer and churn summaries from PostgreSQL.
4. The customer list applies search and risk filters and returns a paginated result.
5. When prediction is requested, Django loads eligible customer, service and billing fields.
6. The trained pipeline generates churn probabilities and maps them to risk categories.
7. Prediction results are saved in PostgreSQL and displayed in the application.

6.3 Simplified architecture diagram

Step	Flow
1	User browser → HTTPS request
2	Render Docker web service → Django views, forms and templates
3	Django ↔ PostgreSQL: customer, service, billing and prediction records
4	Django → trained scikit-learn pipeline: prediction request
5	Pipeline → Django → PostgreSQL: risk score and category
6	Django → browser: dashboard and prediction results

This tabular flow is used instead of a fragile text-art diagram so that the architecture remains readable when the report is opened in different word processors.

7. Database Design

The data model separates customer profile data from service and billing attributes. This reduces repeated fields and allows the application to assemble model features through related records.

Entity	Purpose	Relationship
customers	Customer identifier, profile, demographics and archive state	Main customer record
services	Phone, internet and add-on services	Related to customer
billing_accounts	Tenure, contract, payment method and charges	Related to customer
churn_records	Historical churn label	Related to customer
churn_predictions	Predicted probability, risk category and timestamp	Related to customer
agent_activities	Selected user actions for auditability	Related to application user

7.1 Training data flow

The training module retrieves customers with their service, billing and churn records, builds a feature table, removes the customer identifier from model inputs, and converts the churn label to a binary target. Rows without required related data should be excluded or handled through a documented data-cleaning rule.

7.2 Persistence

PostgreSQL is used for hosted structured data. Because the database is managed separately from the application container, customer records and prediction results remain independent of container restarts and redeployments. Schema migrations and the import of the required customer-related tables must be completed before the hosted workflow is used.

8. Machine-Learning Methodology

8.1 Target and features

The target variable is churn status: Yes is encoded as 1 and No as 0. Customer identifiers are excluded from model features. The remaining inputs include demographic information, service subscriptions, tenure, contract, payment method and charges.

Feature type	Examples	Preprocessing
Numeric	Tenure, monthly charges, total charges, senior-citizen indicator	StandardScaler
Categorical	Contract, internet service, payment method, add-on services	OneHotEncoder with unknown categories ignored
Target	Churn status	Binary label

8.2 Random Forest configuration

Parameter	Configured value
Number of trees	150
Maximum tree depth	10
Minimum samples to split	5
Class weighting	balanced
Random seed	42
Train/test split	90% / 10%, stratified

8.3 Risk classification

Probability	Risk band
0.70 or higher	High
0.35 to below 0.70	Medium
Below 0.35	Low

The thresholds are predefined application rules. They should be validated against model performance and the practical cost of missed churners versus unnecessary retention outreach.

9. Dockerisation and Cloud Deployment

9.1 Deployment design

The application is packaged as a Docker image and deployed as a Render web service. Render provides a public HTTPS endpoint and supports deployments from a Dockerfile. PostgreSQL is hosted separately on Neon so that persistent data does not depend on the lifetime of the web container. This arrangement is straightforward for an academic project and can be operated within free-tier limits, subject to the providers' current terms.

9.2 Docker configuration

The Dockerfile defines a supported Python base image, installs the pinned application requirements, copies the Django source, collects static files, and starts the application server bound to 0.0.0.0 using the platform-provided port. A .dockerignore file excludes virtual environments, Git metadata, cache files, local secrets, and unnecessary generated files.

9.3 Cloud configuration

- Set DEBUG=False and configure an explicit ALLOWED_HOSTS value for the deployed hostname.
- Load the Django secret key and PostgreSQL connection string from environment variables.
- Configure the PostgreSQL connection using the managed database URL.
- Install and pin pandas, scikit-learn, joblib and the PostgreSQL driver alongside Django and other required packages.
- Include the trusted trained model artefact in the image or retrieve it from a versioned trusted location.
- Run database migrations and securely create an administrator account.
- Configure static file collection and verify that styles and scripts load over HTTPS.
- Inspect deployment logs and run smoke tests after each release.

9.4 Deployment workflow

8. Prepare the source repository and remove secrets and local virtual environments.
9. Build the Docker image and test it locally.
10. Create the managed PostgreSQL database and prepare the required schema and data.
11. Set the database URL and Django settings in the hosting platform.
12. Create a Render web service using the Docker runtime and deploy the repository.
13. Verify the public URL, login, dashboard, customer filters and prediction workflow.
14. Restart or redeploy the web service and confirm that PostgreSQL records persist.

9.5 Free-tier limitations

The deployment is intended for a low-traffic course demonstration. Free services may sleep after inactivity, causing a delay on the first request, and have resource or usage limits. These limitations do not prevent a demonstration but mean that the setup should not be described as production-grade high availability. Provider plans can change, so current free-tier limits should be checked before submission.

Official deployment documentation: https://render.com/docs/docker ; https://render.com/docs/free ; https://neon.com/pricing

10. Implementation and Testing

10.1 Application workflow

Django URL patterns route requests to views. Authentication protects customer and dashboard pages. The dashboard aggregates active-customer and churn information, while customer views support searching, filtering and pagination. The prediction module loads the trained pipeline, estimates probability for eligible records, assigns a risk band and stores the result.

10.2 Test plan

Test	**Expected result**
Unauthenticated dashboard request	Redirect to login
Customer search	Matching customer appears
Risk filter	Only records matching the selected risk band appear
Pagination	Large result sets are split into pages
Prediction workflow	Probability and risk category are saved
Invalid or missing model	Clear error is logged; no false success is shown
Database persistence	Records remain after application restart
Cloud URL smoke test	Application responds over HTTPS

10.3 Deployment verification

The deployment should be considered verified only after the public URL has been opened and the main flows tested. Record the deployed URL, date, build status, migration status, prediction test result and screenshots. Model evaluation numbers must come from the actual test run, not from values reported in related research papers.

11. Key Findings and Decisions

11.1 Key findings

- The supplied dataset has 7,043 customer records and 21 columns.
- The observed churn rate is 26.54%, with 1,869 churned and 5,174 retained customers.
- Month-to-month contracts have a higher observed churn rate than longer-term contracts.
- Churn classes are imbalanced, so accuracy alone is an incomplete evaluation metric.
- The project combines a Django user interface with a separate prediction pipeline and relational records.
- Docker and a managed PostgreSQL database make deployment more repeatable and persistent.

11.2 Design decisions

Decision	Reason
Random Forest baseline	Practical model for mixed tabular data
Pipeline-based preprocessing	Keeps training and inference transformations consistent
Separate relational entities	Reduces duplicated customer and service data
Risk bands	Makes probability easier to use in a dashboard
Docker container	Improves portability between development and cloud
Render web service	Simple Docker-based hosting with public HTTPS URL
Managed PostgreSQL	Keeps records independent of the application container
Metrics beyond accuracy	Better evaluation for an imbalanced target

12. Results and Discussion

12.1 Dataset results

Measure	Result
Dataset dimensions	7,043 × 21
Churned customers	1,869 (26.54%)
Retained customers	5,174 (73.46%)
Highest churn contract category	Month-to-month
Classifier configured	Random Forest with 150 trees
Risk levels	High, Medium and Low

12.2 Interpretation

The dataset analysis identifies contract type as a useful segmentation variable for further investigation. The model architecture provides a repeatable way to produce churn-risk estimates, while the dashboard makes those estimates accessible to users. These observations do not establish causality and should not be used toautomatically deny services or apply pricing changes.

12.3 Model performance

Class	Precision	Recall	F1-Score	Support
Retained (0)	0.90	0.77	0.83	518
Churned (1)	0.55	0.76	0.64	187
Accuracy			0.77	705
Macro Average	0.72	0.77	0.73	705
Weighted Average	0.81	0.77	0.78	705

ROC-AUC Score: 0.8378

12.4 Screen shots

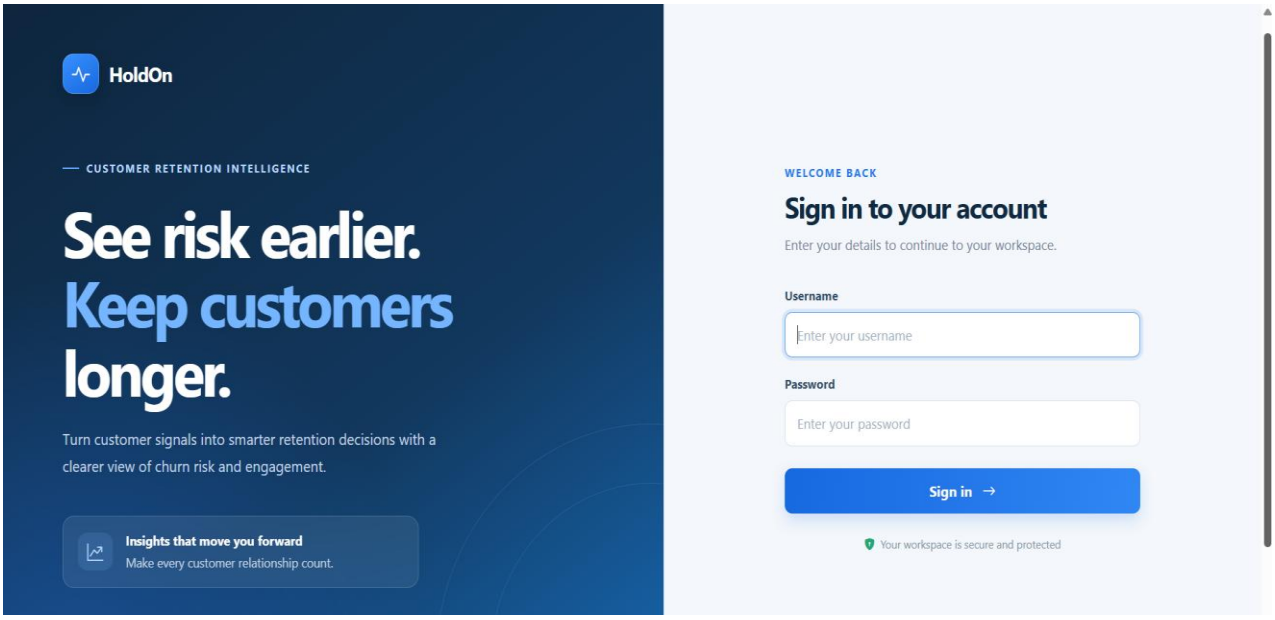


Figure 12.1: Login Interface.

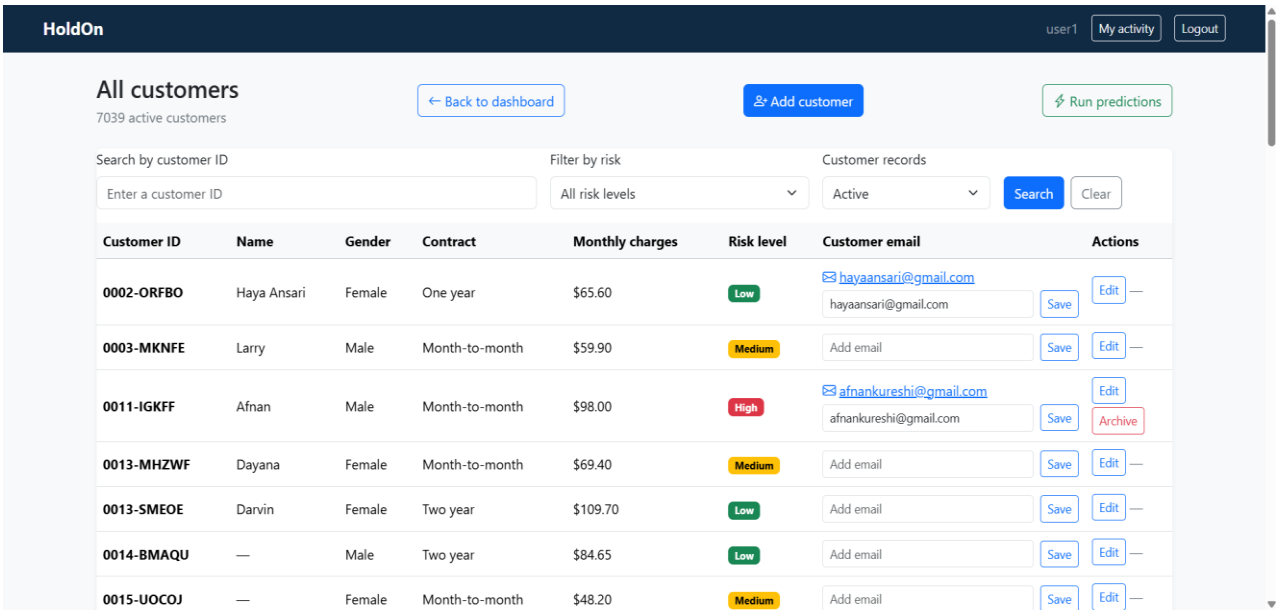


Figure 12.2: Customer List with Risk Badges

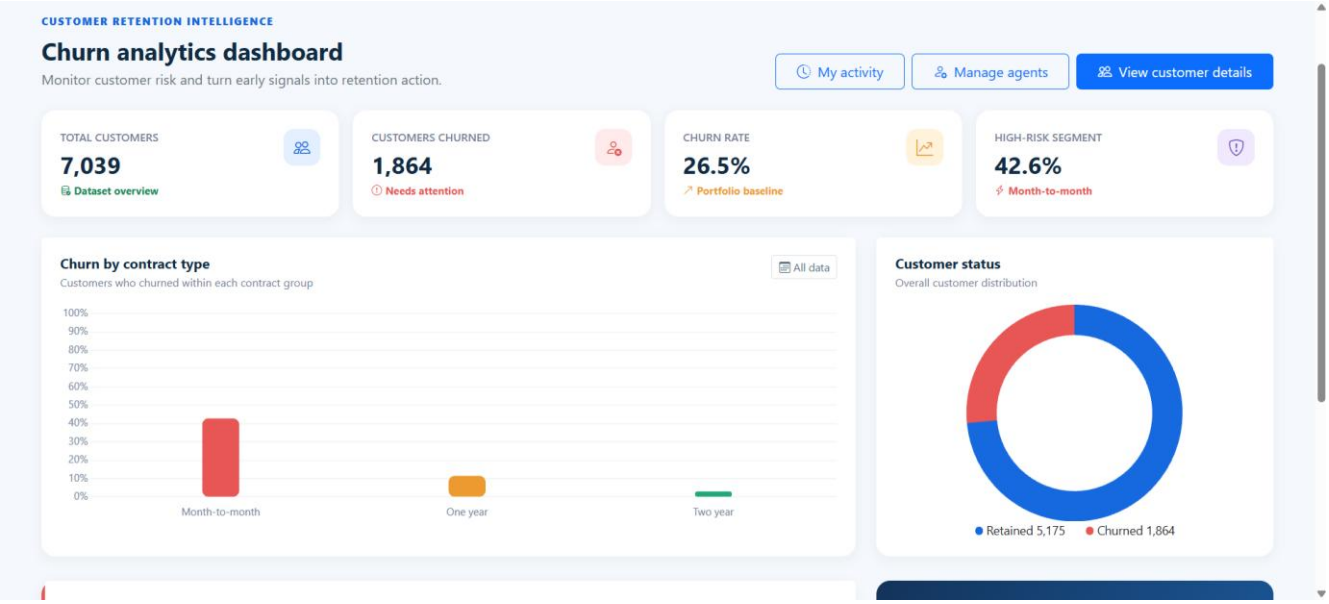


Figure 12.3: User Dashboard Showing the customer risk analytics

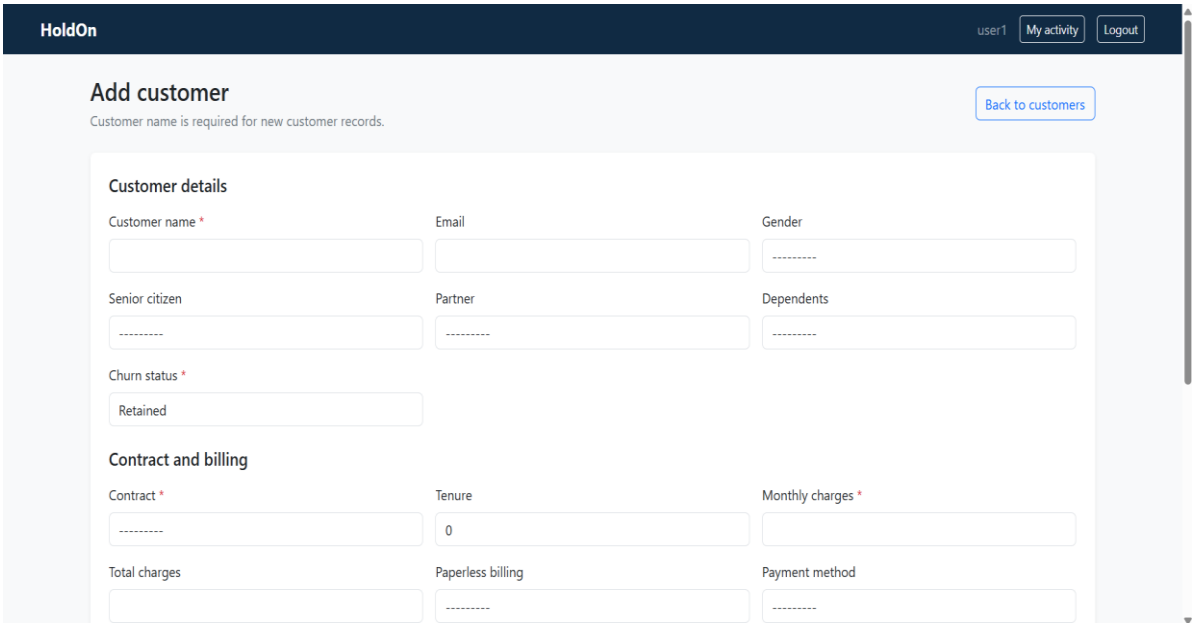


Figure 12.4: Form for adding or editing a customer, with billing and service details

13. Security, Limitations and Future Work

13.1 Security considerations

- Store secret keys and database credentials in environment variables, not source code.
- Disable debug mode and restrict allowed hosts in the hosted environment.
- Use HTTPS and appropriate secure session and CSRF settings.
- Use strong administrator credentials and least-privilege database access.
- Validate user input and protect state-changing requests with CSRF controls.
- Avoid storing unnecessary personal data and never expose credentials in logs.

13.2 Limitations

- The benchmark dataset may not represent current customer behaviour at a real operator.
- Risk thresholds are predefined and may not be optimal for a real retention campaign.
- Free hosting may have cold starts, resource limits and no production availability guarantee.

13.3 Future work

- Compare Random Forest with logistic regression, XGBoost or LightGBM.
- Add feature explanations to help users understand risk estimates.
- Track model drift and retrain using recent, representative data.
- Add automated tests and continuous deployment checks.
- Measure whether retention actions based on model output actually reduce churn.

14. Conclusion

This project integrates customer data management, a web dashboard and a Random Forest churn-prediction workflow in a Django application. The supplied dataset contains 7,043 customer records, of which 26.54% are labelled as churned. Exploratory results show a higher observed churn rate among month-to-month customers, illustrating how analytics can help identify customer groups for further investigation.

The cloud-computing component uses Docker to package the application, Render to host the web service, and managed PostgreSQL to store persistent records. The architecture separates application runtime from the database and supports reproducible deployment. The implementation also illustrates why environment-based configuration, data validation, model evaluation and smoke testing are essential when moving a local application to the cloud.

The project provides a foundation for customer retention analysis, but its predictions must be evaluated with a held-out test set and interpreted carefully. Future improvements include model comparison, explainability, automated testing and monitoring. Together, the application and deployment design demonstrate the practical integration of cloud computing, database systems, web development and machine learning.

References

1. Lalwani, P., Mishra, M. K., Chadha, J. S., & Sethi, P. (2021). Customer churn prediction system: a machine learning approach. Computing, 104, 271–294. https://doi.org/10.1007/s00607-021-00908-y

2. Vafeiadis, T., Diamantaras, K. I., Sarigiannidis, G., & Chatzisavvas, K. C. (2015). A comparison of machine learning techniques for customer churn prediction. Simulation Modelling Practice and Theory, 55, 1–9.

3. Verbeke, W., Martens, D., Mues, C., & Baesens, B. (2011). Building comprehensible customer churn prediction models with advanced rule induction techniques. Expert Systems with Applications, 38, 2354–2364.

4. Xie, Y., Li, X., Ngai, E. W. T., & Ying, W. (2009). Customer churn prediction using improved balanced random forests. Expert Systems with Applications, 36, 5445–5449.

5. Imani, M., Joudaki, M., Beikmohammadi, A., & Arabnia, H. R. (2025). Customer churn prediction: A systematic review of recent advances, trends, and challenges in machine learning and deep learning. *Systematic Review*, 1–15.

6. scikit-learn documentation. https://scikit-learn.org/stable/

7. Docker documentation. https://docs.docker.com/

8. Render documentation: Docker deployments and free services. https://render.com/docs/docker ; https://render.com/docs/free

9. Neon pricing and plans. https://neon.com/pricing

10. Uploaded dataset: WA_Fn-UseC_-Telco-Customer-Churn.csv.